

# ARCHITECTURAL MANUAL

---

HTTPIE

# Executive System Vision

---

## Chapter 1: Executive System Vision

### 1.1 Business Logic and Domain-Driven Design

The `httpie` repository employs a domain-driven design approach to organize its business logic. The core value of the system lies in its ability to parse and process HTTP requests. The `httpie` parser is designed to handle various types of input, including JSON, forms, and file uploads.

The `KeyValueArg` class is a fundamental component of the parser, representing a key-value pair extracted from the input. The `process_query_param_arg`, `process_embed_query_param_arg`, and `process_file_upload_arg` functions demonstrate how the system handles different types of input.

```
def process_query_param_arg(arg: KeyValueArg) -> str:
    return arg.value

def process_embed_query_param_arg(arg: KeyValueArg) -> str:
    return load_text_file(arg).rstrip('\n')

def process_file_upload_arg(arg: KeyValueArg) -> Tuple[str, IO, str]:
    parts = arg.value.split(SEPARATOR_FILE_UPLOAD_TYPE)
    filename = parts[0]
    mime_type = parts[1] if len(parts) > 1 else None
    try:
        f = open(os.path.expanduser(filename), 'rb')
    except OSError as e:
        raise ParseError(f'{arg.orig!r}: {e}')
    return (os.path.basename(filename), f, mime_type or
            get_content_type(filename))
```

### 1.2 Core Value and System Vision

The core value of the `httpie` system is its ability to provide a simple and intuitive way to send HTTP requests. The system vision is to make it easy for users to interact with web services and APIs.

The `httpie` parser is designed to handle various types of input, including JSON, forms, and file uploads. The system uses a domain-driven design approach to organize its business logic, with a focus on parsing and processing HTTP requests.

```
graph LR
    A[Input] -->|JSON| B[JSON Parser]
    A -->|Form| C[Form Parser]
    A -->|File Upload| D[File Upload Parser]
    B -->|Parsed JSON| E[Request Builder]
    C -->|Parsed Form| E
    D -->|Parsed File Upload| E
```

```
E -->|HTTP Request|> F[HTTP Client]
F -->|HTTP Response|> G[Response Handler]
```

### 1.3 Color Palette and Theme

The `httpie` system uses a color palette and theme to provide a visually appealing and consistent user experience. The `GenericColor` enum defines a set of generic colors that are safe to use everywhere.

```
class GenericColor(Enum):
    WHITE = {Styles.PIE: PieColor.WHITE, Styles.ANSI: 'white'}
    BLACK = {Styles.PIE: PieColor.BLACK, Styles.ANSI: 'black'}
    GREEN = {Styles.PIE: PieColor.GREEN, Styles.ANSI: 'green'}
    ORANGE = {Styles.PIE: PieColor.ORANGE, Styles.ANSI: 'yellow'}
    YELLOW = {Styles.PIE: PieColor.YELLOW, Styles.ANSI: 'bright_yellow'}
    BLUE = {Styles.PIE: PieColor.BLUE, Styles.ANSI: 'blue'}
    PINK = {Styles.PIE: PieColor.PINK, Styles.ANSI: 'bright_magenta'}
    PURPLE = {Styles.PIE: PieColor.PURPLE, Styles.ANSI: 'magenta'}
    RED = {Styles.PIE: PieColor.RED, Styles.ANSI: 'red'}
    AQUA = {Styles.PIE: PieColor.AQUA, Styles.ANSI: 'cyan'}
    GREY = {Styles.PIE: PieColor.GREY, Styles.ANSI: 'bright_black'}
```

The `Theme` class is used to create a theme that applies the color palette to the system.

```
class Theme:
    def __init__(self):
        self.styles = {}
        for color, color_set in ChainMap(GenericColor.__members__,
CUSTOM_STYLES).items():
            if isinstance(color_set, _StyledGenericColor):
                properties = dict.fromkeys(color_set.styles, True)
                color_set = color_set.color
            else:
                properties = {}
            self.styles[color.lower()] = Style(color=color_set.apply_style(style,
style_name=style_name), **properties)
```

### 1.4 Conclusion

In conclusion, the `httpie` system employs a domain-driven design approach to organize its business logic, with a focus on parsing and processing HTTP requests. The system uses a color palette and theme to provide a visually appealing and consistent user experience. The `KeyValueArg` class and `process_query_param_arg`, `process_embed_query_param_arg`, and `process_file_upload_arg` functions demonstrate how the system handles different types of input.

# Architectural Design Patterns

---

## Chapter 2: Architectural Design Patterns

### 2.1 Singleton Pattern

The Singleton pattern is a creational design pattern that restricts the instantiation of a class to a single instance. This is useful when exactly one object is needed to coordinate actions across the system.

In the `httpie` repository, the `PluginManager` class is an example of the Singleton pattern. The `plugin_manager` instance is created and registered with all built-in plugins.

```
plugin_manager = PluginManager()
plugin_manager.register(
    BasicAuthPlugin, DigestAuthPlugin, BearerAuthPlugin,
    HeadersFormatter, JSONFormatter, XMLFormatter, ColorFormatter,
)
```

The `PluginManager` class is designed to manage plugins, and it is not intended to be instantiated multiple times. The `plugin_manager` instance is used throughout the codebase to access and manage plugins.

### 2.2 Factory Pattern

The Factory pattern is a creational design pattern that provides a way to create objects without specifying the exact class of object that will be created.

In the `httpie` repository, the `PluginManager` class uses the Factory pattern to create instances of plugins. The `get_transport_plugins` method returns a list of transport plugin classes, which can be instantiated later.

```
def get_transport_plugins(self) -> List[Type[TransportPlugin]]:
    return self.filter(TransportPlugin)
```

The `filter` method is used to filter plugins based on their type, and it returns a list of plugin classes that can be instantiated later.

### 2.3 Service Layer Pattern

The Service Layer pattern is an architectural pattern that defines a layer of abstraction between the business logic and the presentation layer.

In the `httpie` repository, the `PluginManager` class can be seen as a service layer that provides a way to access and manage plugins. The `PluginManager` class encapsulates the logic of plugin management and provides a simple interface for accessing plugins.

```
def get_auth_plugins(self) -> List[Type[AuthPlugin]]:
    return self.filter(AuthPlugin)
```

The `get_auth_plugins` method is an example of a service layer method that provides a way to access authentication plugins.

## 2.4 Dependency Injection Pattern

The Dependency Injection pattern is a design pattern that allows components to be loosely coupled, making it easier to test and maintain the system.

In the `httpie` repository, the `PluginManager` class uses dependency injection to manage plugins. The `register` method is used to register plugins with the `PluginManager` instance.

```
def register(self, *plugins):
    for plugin in plugins:
        self.plugins.append(plugin)
```

The `register` method is an example of dependency injection, where plugins are injected into the `PluginManager` instance.

## Logic Flow Diagram

```
graph LR
    A[PluginManager] -->|register|> B[Plugin]
    A -->|get_transport_plugins|> C[TransportPlugin]
    A -->|get_auth_plugins|> D[AuthPlugin]
    C -->|filter|> E[TransportPlugin]
    D -->|filter|> F[AuthPlugin]
```

This diagram shows the logic flow of the `PluginManager` class, including the registration of plugins, the retrieval of transport plugins, and the retrieval of authentication plugins.

## Conclusion

In conclusion, the `httpie` repository uses various architectural design patterns, including the Singleton pattern, Factory pattern, Service Layer pattern, and Dependency Injection pattern. These patterns help to make the codebase more maintainable, testable, and scalable. The `PluginManager` class is a key component of the `httpie` repository, and it uses these patterns to manage plugins and provide a simple interface for accessing plugins.

# Data Flow & Persistence

---

## Chapter 3: Data Flow & Persistence

### 3.1 Schema Design

The `httpie` repository utilizes a YAML file (`methods.yaml`) to store its database. The schema design is implicit, meaning it is defined by the structure of the data stored in the YAML file. The `load_database` function reads the YAML file and returns a Python dictionary representing the database.

```
def load_database() -> Database:
    return yaml.safe_load(DB_FILE.read_text(encoding='utf-8'))
```

The database schema is composed of several key-value pairs, where each key represents a specific entity or concept in the `httpie` domain. For example, the `KEY_DOC_STRUCTURE` key stores the documentation structure, while the `KEY_TOOLS` key stores information about the tools used in the project.

### 3.2 ORM Usage

The `httpie` repository does not use an Object-Relational Mapping (ORM) system in the classical sense. Instead, it relies on the `yaml` library to serialize and deserialize data between the YAML file and Python dictionaries. This approach is often referred to as a "lightweight ORM" or "dictionary-based ORM."

However, the repository does define several data classes and functions that provide a layer of abstraction between the raw YAML data and the application logic. For example, the `Group` class represents a group of arguments, and the `Argument` class represents a single argument.

```
@dataclass
class Group:
    name: str
    description: str = ''
    is_mutually_exclusive: bool = False
    arguments: List['Argument'] = field(default_factory=list)

class Argument(typing.NamedTuple):
    aliases: List[str]
    configuration: Dict[str, Any]
```

These data classes and functions can be seen as a form of ORM, as they provide a structured way of accessing and manipulating the data stored in the YAML file.

### 3.3 Database Transaction Logic

The `httpie` repository does not implement traditional database transactions, as it does not use a relational database management system. Instead, it relies on the `yaml` library to read and write data to the YAML file.

However, the repository does implement a form of transaction logic through the use of context managers. For example, the `on_repo` context manager is used to create an isolated instance of the repository, where changes can be made without affecting the original repository.

```
@contextmanager
def on_repo(self) -> Generator[Tuple[Path, Dict[str, str]], None, None]:
    """ Return the path to the python interpreter and the environment variables
    (e.g HTTPIE_COMMAND) to be used on the benchmarks. """
    raise NotImplementedError
```

This approach provides a way to manage changes to the repository in a transactional manner, ensuring that changes are either committed or rolled back as a single unit.

### 3.4 Mermaid.js Diagram

```
graph LR
    A[YAML File] -->|read| B[Python Dictionary]
    B -->|deserialize| C[Data Classes]
    C -->|access/manipulate| D[Application Logic]
    D -->|serialize| E[Python Dictionary]
    E -->|write| A
```

This diagram shows the flow of data between the YAML file, Python dictionaries, data classes, and application logic. The `yaml` library is used to read and write data to the YAML file, while the data classes provide a structured way of accessing and manipulating the data.

### 3.5 Conclusion

In conclusion, the `httpie` repository implements a lightweight ORM system through the use of data classes and functions that provide a layer of abstraction between the raw YAML data and the application logic. The repository also implements a form of transaction logic through the use of context managers, ensuring that changes to the repository are managed in a transactional manner.

# API & Interface Strategy

---

## 4.1 API & Interface Strategy Overview

The `httpie` repository implements a robust API and interface strategy to handle various aspects of HTTP requests and responses. This chapter delves into the specifics of how `httpie` handles REST/GraphQL constraints, serialization, and endpoint security.

### 4.1.1 REST/GraphQL Constraints

`httpie` uses the `requests` library to handle HTTP requests. The `requests` library provides a simple and intuitive API for making HTTP requests. However, `httpie` also supports GraphQL queries, which require a different set of constraints.

To handle these constraints, `httpie` uses the `graphql` library, which provides a Python implementation of the GraphQL specification. The `graphql` library allows `httpie` to parse and execute GraphQL queries.

### 4.1.2 Serialization

`httpie` uses the `json` library to serialize and deserialize JSON data. The `json` library provides a simple and efficient way to convert between JSON data and Python objects.

However, `httpie` also supports other serialization formats, such as XML and URL-encoded data. To handle these formats, `httpie` uses the `xmltodict` library to parse XML data and the `urllib.parse` library to parse URL-encoded data.

### 4.1.3 Endpoint Security

`httpie` provides several features to ensure endpoint security, including:

- **Authentication:** `httpie` supports various authentication mechanisms, including Basic Auth, Digest Auth, and OAuth.
- **SSL/TLS verification:** `httpie` verifies the SSL/TLS certificates of the endpoints it connects to, ensuring that the connection is secure.
- **Data encryption:** `httpie` encrypts data in transit using SSL/TLS, ensuring that the data is protected from eavesdropping and tampering.

### 4.1.4 Code Organization

The `httpie` repository is organized into several modules, each responsible for a specific aspect of the API and interface strategy. The main modules are:

- `httpie.core`: This module contains the core logic of `httpie`, including the API and interface strategy.
- `httpie.plugins`: This module contains plugins that extend the functionality of `httpie`.
- `httpie.utils`: This module contains utility functions used throughout the `httpie` repository.

### 4.1.5 Mermaid.js Diagram



```
graph LR
  A[httpie.core] -->|uses| B[requests]
  A -->|uses| C[graphql]
  A -->|uses| D[json]
  A -->|uses| E[xmltodict]
  A -->|uses| F[urllib.parse]
  B -->|makes| G[HTTP requests]
  C -->|parses| H[GraphQL queries]
  D -->|serializes| I[JSON data]
  E -->|parses| J[XML data]
  F -->|parses| K[URL-encoded data]
  G -->|sends| L[requests]
  H -->|executes| M[GraphQL queries]
  I -->|deserializes| N[JSON data]
  J -->|deserializes| O[XML data]
  K -->|deserializes| P[URL-encoded data]
  L -->|receives| Q[responses]
  M -->|returns| R[results]
  N -->|returns| S[deserialized data]
  O -->|returns| T[deserialized data]
  P -->|returns| U[deserialized data]
  Q -->|returns| V[responses]
  R -->|returns| W[results]
  S -->|returns| X[deserialized data]
  T -->|returns| Y[deserialized data]
  U -->|returns| Z[deserialized data]
  V -->|returns| AA[responses]
  W -->|returns| BB[results]
  X -->|returns| CC[deserialized data]
  Y -->|returns| DD[deserialized data]
  Z -->|returns| EE[deserialized data]
  AA -->|returns| FF[responses]
  BB -->|returns| GG[results]
  CC -->|returns| HH[deserialized data]
  DD -->|returns| II[deserialized data]
  EE -->|returns| JJ[deserialized data]
  FF -->|returns| KK[responses]
  GG -->|returns| LL[results]
  HH -->|returns| MM[deserialized data]
  II -->|returns| NN[deserialized data]
  JJ -->|returns| OO[deserialized data]
  KK -->|returns| PP[responses]
  LL -->|returns| QQ[results]
```

```
# Resilience & QA Strategy
```

```
# CHAPTER 5: Resilience & QA Strategy
```

```
## 5.1 Error Handling
```

Error handling is a critical aspect of any robust application. In the context of the `httpie` repository, error handling is implemented through a combination of try-except blocks, custom error classes, and logging mechanisms.

The `handle_generic_error` function is a central component of the error handling mechanism. It takes an exception object `e` and an optional annotation string as input. The function extracts relevant information from the exception object, such as the request URL and method, and constructs an error message. The error message is then logged using the `env.log_error` method.

```
```python
def handle_generic_error(e, annotation=None):
    msg = str(e)
    if hasattr(e, 'request'):
        request = e.request
        if hasattr(request, 'url'):
            msg = (f'{msg} while doing a {request.method} request to URL:
{request.url}')
    if annotation:
        msg += annotation
    env.log_error(f'{type(e).__name__}: {msg}')
    if include_traceback:
        raise
    if include_debug_info:
        print_debug_info(env)
    if args == ['--debug']:
        return ExitStatus.SUCCESS
    exit_status = ExitStatus.SUCCESS
```

In addition to the `handle_generic_error` function, the `httpie` repository defines several custom error classes, such as `ConnectionError` and `NestedJSONSyntaxError`. These classes inherit from the base `Exception` class and provide additional context and information about the error.

```
class ConnectionError(Exception):
    pass

class NestedJSONSyntaxError(Exception):
    pass
```

## 5.2 Unit Test Coverage

Unit testing is an essential aspect of ensuring the quality and reliability of the `httpie` repository. The repository uses the `pytest` framework to write and run unit tests.

The `test_error` function is an example of a unit test that verifies the error handling mechanism. The test simulates a `ConnectionError` exception and verifies that the error message is correctly logged and that the exit status is set to `ExitStatus.ERROR`.

```
@mock.patch('httpie.core.program')
def test_error(program):
    exc = ConnectionError('Connection aborted')
    exc.request = Request(method='GET', url='http://www.google.com')
    program.side_effect = exc
    r = http('www.google.com', tolerate_error_exit_status=True)
    assert r.exit_status == ExitStatus.ERROR
    error_msg = ('ConnectionError: Connection aborted while doing a GET request to
URL: http://www.google.com')
    assert error_msg in r.stderr
```

## 5.3 Fault Tolerance

Fault tolerance is the ability of a system to continue operating even when one or more components fail. In the context of the `httpie` repository, fault tolerance is achieved through the use of try-except blocks and error handling mechanisms.

The `test_keyboard_interrupt_during_arg_parsing_exit_status` function is an example of a unit test that verifies the fault tolerance of the `httpie` repository. The test simulates a `KeyboardInterrupt` exception during argument parsing and verifies that the exit status is set to `ExitStatus.ERROR_CTRL_C`.

```
def test_keyboard_interrupt_during_arg_parsing_exit_status(httpbin):
    with mock.patch('httpie.cli.definition.parser.parse_args',
side_effect=KeyboardInterrupt()):
        r = http('GET', httpbin + '/get', tolerate_error_exit_status=True)
        assert r.exit_status == ExitStatus.ERROR_CTRL_C
```

## 5.4 Logic Flow Diagram

The following Mermaid.js diagram illustrates the logic flow of the error handling mechanism in the `httpie` repository:

```
graph LR
    A[Error Occurs] -->|try-except block| B[Handle Generic Error]
    B -->|log error message| C[Log Error]
    C -->|set exit status| D[Set Exit Status]
    D -->|return exit status| E[Return Exit Status]
```

```
E -->|optional: raise exception|> F[Raise Exception]
F -->|optional: print debug info|> G[Print Debug Info]
```

In this diagram, the error handling mechanism is triggered when an error occurs (A). The error is then handled by the `handle_generic_error` function (B), which logs the error message (C) and sets the exit status (D). The exit status is then returned (E), and optionally, the exception is raised (F) and debug information is printed (G).

# DevOps & Deployment Topology

---

## Chapter 6: DevOps & Deployment Topology

### 6.1 CI/CD Pipelines

The `httpie` repository utilizes a CI/CD pipeline to automate testing, building, and deployment of the project. The pipeline is configured using GitHub Actions, which provides a flexible and scalable way to automate workflows.

The pipeline is defined in the `.github/workflows/main.yml` file, which specifies the jobs to be executed, the environment, and the steps to be taken. The pipeline consists of two main jobs: `build` and `test`.

The `build` job is responsible for building the `httpie` wheel distribution. It uses the `setup.py` file to create a wheel distribution of the project, which can be installed using `pip`.

The `test` job is responsible for running the tests for the project. It uses the `pytest` framework to run the tests, which are defined in the `tests` directory.

### 6.2 Containerization

The `httpie` repository uses containerization to ensure consistent and reproducible builds. The `Dockerfile` defines the environment and dependencies required to build and run the project.

The `Dockerfile` uses the `python:3.9-slim` image as the base image, which provides a minimal Python 3.9 environment. The image is then customized by installing the required dependencies, including `pip`, `setuptools`, and `wheel`.

The `docker-compose.yml` file defines the services required to build and run the project. The `httpie` service uses the `Dockerfile` to build the image, and the `test` service uses the `pytest` framework to run the tests.

### 6.3 Environment Configuration

The `httpie` repository uses environment variables to configure the project. The `Environment` class defines the environment variables required to build and run the project.

The `Environment` class uses the `pathlib` library to define the paths to the project directories. The `config` attribute defines the path to the configuration file, which is used to store the project settings.

The `Config` class defines the configuration settings for the project. The `Config` class uses the `json` library to load and save the configuration settings.

### 6.4 Deployment Topology

The `httpie` repository uses a deployment topology to ensure consistent and reproducible deployments. The deployment topology is defined in the `docs/deployment.md` file, which specifies the steps required to deploy the project.

The deployment topology consists of the following steps:

1. Build the wheel distribution of the project using the `setup.py` file.
2. Install the wheel distribution using pip.
3. Configure the project settings using the `Config` class.
4. Run the tests using the `pytest` framework.

## Mermaid.js Diagram

```
graph LR
  A[CI/CD Pipeline] -->|build| B[Build Wheel Distribution]
  A -->|test| C[Run Tests]
  B -->|install| D[Install Wheel Distribution]
  C -->|configure| E[Configure Project Settings]
  D -->|run| F[Run Project]
  E -->|run| F
```

## Code Snippets

```
# setup.py
from setuptools import setup

setup(
    name='httpie',
    version='1.0.0',
    packages=['httpie'],
    install_requires=['requests'],
    entry_points={
        'console_scripts': ['httpie=httpie.cli:main'],
    },
)
```

```
# httpie/cli.py
import argparse
import sys

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--version', action='version', version='%(prog)s 1.0.0')
    args = parser.parse_args()
    print('Hello, World!')

if __name__ == '__main__':
    sys.exit(main())
```

```
# tests/test_httpie.py
import pytest
```

```
from httpie import cli

def test_httpie():
    assert cli.main(['--version']) == 0
```

```
# Dockerfile
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

CMD ["httpie", "--version"]
```

```
# docker-compose.yml
version: '3'

services:
  httpie:
    build: .
    ports:
      - "8080:8080"
    depends_on:
      - test
    environment:
      - PYTHONDONTWRITEBYTECODE=1
      - PYTHONUNBUFFERED=1

  test:
    build: .
    command: pytest
    environment:
      - PYTHONDONTWRITEBYTECODE=1
      - PYTHONUNBUFFERED=1
```

```
# Environment.py
import os
from pathlib import Path

class Environment:
    def __init__(self):
```

